Automata & ANTLR Cheat Sheet

Comprehensive guide covering formal language theory, automata models, grammar formalisms, and ANTLR usage with rich examples.

1. Fundamentals of Formal Languages

1.1 Basic Definitions

- Alphabet (Σ): A finite set of symbols. Example: Σ = {a, b, 0, 1, +, *}.
- String (word): A finite sequence of symbols from Σ. Example: abba, 1010, a+bb*.
- Empty string (ε / λ): The unique string of length zero.
- Kleene closure (Σ*): The set of all strings (including ε) over Σ.
- Language: Any subset of Σ*. Example: L = { w ∈ {a,b}* : w contains an even number of a's }.

1.2 Operations on Languages

- Union ($L_1 \cup L_2$): All strings in L_1 or L_2.
- Concatenation ($L_1 \cdot L_2$): All strings formed by x from L_1 followed by y from L_2.
- Kleene star (L*): Zero or more concatenations of strings from L.
- Intersection / Complement: Defined when Σ is fixed, used for closure properties.

1.3 Example: Even-a Language

Σ = {a, b} L = { w | w has an even number of a's } Regular expression: (b*a*ba)*b* DFA states encode parity: Q = {q_even, q_odd}, start = q_even, accept = {q_even}.

1. Regular Languages & Regular Expressions

2.1 Regular Expressions Syntax

Operator	Syntax	Meaning	Example
Concatenation	r_1r_2	Strings matching r_1 followed by r_2	ab
Union	$r_1 \mid r_2$	Strings matching r_1 or r_2	a | b
Kleene star	r*	Zero or more repetitions of r	a*
Plus	r+	One or more repetitions (r r*)	a+
Optional	r?	Zero or one occurrence	a?
Character class	[abc]	Any one of a, b, or c	[0-9] matches any digit
Negation	[^0-9]	Any symbol not in the class	[^ab]

2.2 Example Patterns

1. Binary strings containing "101": (0|1)*101(0|1)*

2. Strings of a's and b's with odd length: ((a|b)(a|b))* (a|b)

3. Floating-point numbers: [+-]?([0-9]+.[0-9]*|.[0-9]+)([eE][+-]?[0-9]+)?

4. Identifiers in many languages: [a-zA-Z_][a-zA-Z_0-9]*

5. Finite Automata Finite Automata

3.1 Deterministic Finite Automaton (DFA)

Definition: (Q, Σ, δ, q_0, F). Transition δ: Q × Σ → Q. Example: Language of binary strings with even number of 1's. Q = {q_0 (even), q_1 (odd)}; Σ={0,1} δ(q_0,1)=q_1; δ(q_1,1)=q_0; δ(_,0)=same state; start=q_0; accept={q_0}.

3.2 Nondeterministic Finite Automaton (NFA)

Definition: δ: Q × (Σ ∪ {ε}) → 2^Q. Accept if any path leads to accepting state. ε-transitions allow spontaneous moves. Subset construction yields equivalent DFA.

3.3 Pumping Lemma for Regular Languages

Statement: ∃ p such that any w ∈ L with |w| ≥ p can be decomposed w=xyz satisfying:

- |xy| ≤ p
- |y| ≥ 1
- ∀ i ≥ 0, x y^i z ∈ L Use: Prove non-regularity. Example: L={a^n b^n | n≥0} is not regular by contradiction.

- Context-Free Grammars (CFG)

In simple terms, a context-free grammar is a set of rules for building strings by starting from one symbol and repeatedly replacing symbols according to those rules.

4.1 What's in a CFG?

- **Nonterminals** (often uppercase letters like S, A, B): placeholders that need to be replaced.
- **Terminals** (symbols from your alphabet, like `a`, `b`, `(`, `)`): the actual characters in your strings.
- **Start symbol** (usually S): where you begin generating.
- **Productions**: rules of the form A → α, meaning "replace A with the string α." α can mix terminals and nonterminals.

4.2 How to build a CFG: step by step

1. **Identify the pattern** in your language (e.g., matching numbers of symbols, nesting, choices).
2. **Pick nonterminals** for each chunk or subpattern (e.g., use P for parentheses, or S for the whole string).

3. **Write rules** that either:
4. **Expand** a nonterminal into smaller parts (recursion), or
5. **Finish** with terminals (base cases).

4.3 Easy examples

Example A: Balanced parentheses

Language: all strings of `(` and `)` that nest correctly, like `()`, `(())()`, but not `)(`.

```
S → S S       // you can put two valid pieces next to each other
S → '(' S ')' // or wrap a valid piece in parentheses
S → ε         // or empty string is valid
```

- Start at S. Either concatenate two S's, or add one pairing of parentheses around an S, or stop with empty.

Example B: Palindromes over {a, b}

Language: same forwards and backwards, e.g. `abba`, `aba`, `b`.

```
P → 'a' P 'a'  // put 'a' at both ends
P → 'b' P 'b'  // put 'b' at both ends
P → 'a'        // single 'a'
P → 'b'        // single 'b'
P → ε          // or empty
```

- Each step adds matching symbols around the smaller palindrome.

Example C: Simple arithmetic expressions

Language: addition and multiplication with numbers (no precedence handled here).

```
E → E '+' E    // addition
E → E '*' E    // multiplication
E → 'n'        // a number (placeholder 'n')
E → '(' E ')'  // parentheses
```

- `E` can combine two expressions with `+` or `*`, or be a number, or be an expression in parentheses.

4.4 More tailored example

Language: strings of a's and b's where number of b's is twice number of a's

We want L = { a^n b^m : m = 2n }.

1. Each `a` should correspond to two `b`s. We use recursion:

```
S → 'a' S 'b' 'b'  // for each 'a', add two 'b's later
S → ε              // stop when no more 'a's
```

- If n=3, expansions could be: S → 'a' S 'b' 'b' → 'a' 'a' S 'b' 'b' 'b' 'b' → 'a' 'a' 'a' S 'b' 'b' 'b' 'b' 'b' 'b' → 'a''a''a''ε''b''b''b''b''b''b' = `aaabbbbbb`.

4.5 Quick tips

- **Recursion** (rules that refer back to the same nonterminal) is how you handle repetition or matching counts.
- **Base cases** (rules that only have terminals or ε) stop the recursion.
- Draw a small parse tree on paper: each rule application is a branch.
- ANTLR Overview & Workflow

6.1 What is ANTLR?

ANother Tool for Language Recognition. Generates: Lexer, Parser, Listener, Visitor. Supports actions/ semantic predicates, error recovery.

6.2 Typical Workflow

1. Define grammar in .g4 file (parser + lexer rules).
2. Generate code with antlr4 tool.
3. Parse input to build parse tree.
4. Traverse tree using listener or visitor to build AST or interpret.
5. Embed semantic actions or use listener/visitor to enforce semantics.
6. Writing ANTLR Grammars (.g4 files)

7.1 Lexer vs Parser Rules

- Lexer rules: Uppercase names, define tokens (e.g. INT : [0-9]+;).
- Parser rules: Lowercase, define language structure (e.g. expr : expr '+' term;).

7.2 Common Patterns

```
WS : [ \t\r\n]+ -> skip;
LINE_COMMENT : '//' ~[\r\n]* -> skip;
PRINT : 'print';
ID : [a-zA-Z_] [a-zA-Z_0-9]*;
INT : [0-9]+;
```

7.3 Expression Grammar with Precedence

```
grammar Expr;
prog : stat+ ;
stat : ID '=' expr ';' | 'print' expr ';' ;
expr : <assoc=right> expr '^' expr # Pow
    | expr '*' expr          # Mul
    | expr '+' expr          # Add
    | INT                    # Int
    | ID                     # Id
    | '(' expr ')'           # Parens
;
```

7.4 Multiple Assignment & Lists

```
stat : vars '=' exprs ';' ;
vars : ID (',' ID)* ;
exprs: expr (',' expr)* ;
```

1. Tree Walking Strategies

8.1 Listeners (Event-Driven)

Generated BaseListener with empty enter/exit methods. Override enterRule or exitRule callbacks. Use case: Logging, building AST with external data structures.

8.2 Visitors (Return-Driven)

Generated BaseVisitor with visitX(Context) for each rule. Override visit methods, return computed values. Use case: Expression evaluation, AST transformation.

8.3 Example: Expression Visitor (Java)

```
@Override
public Integer visitAdd(ExprParser.AddContext ctx) {
    int left = visit(ctx.expr(0));
    int right = visit(ctx.expr(1));
```

```
    return left + right;
}

@Override
public Integer visitInt(ExprParser.IntContext ctx) {
    return Integer.parseInt(ctx.INT().getText());
}
```

1. Error Handling & Debugging

2. Error listeners: Implement ANTLRErrorListener to catch syntax errors.

3. Diagnostic error listener: parser.addErrorListener(new DiagnosticErrorListener());
4. Bail strategy: parser.setErrorHandler(new BailErrorStrategy());

5. Visualizing parse trees: Use ANTLR's TestRig (grun) or IDE plugins.

6. Common Pitfalls & Tips

7. Left recursion: Convert to iterative form or use ANTLR's automatic elimination.

8. Grammar ambiguity: Refactor with precedence/associativity.
9. Token conflicts: Order matters; place keywords before identifiers.
10. Hidden tokens: Use channels to skip whitespace/comments.

11. Performance: Avoid excessive backtracking; use diagnostic listeners to detect.

12. Constructing Grammars: Strategy & Examples

13.1 General Strategy

- For **regular grammars**:

- Design a DFA that recognizes the language (states capture needed counts or parities).

- Convert each transition q --x--> r into a production: Nonterminal_q → x Nonterminal_r.

- Mark productions to ε (empty) for any accepting DFA state.

- For **context-free grammars**:

- Identify recursive patterns (e.g. matching a's with b's).

- Introduce nonterminals for recursion and base cases.
- Write productions that reflect the required relationships (e.g. equal counts, offsets).

13.2 Example 1: Regular Grammar for L = { w ∈ {a,b}* : na(w) ≥ 3, nb(w) odd }

1. **Build DFA** with states tracking (#a's seen: 0,1,2,≥3) × (b-parity: even, odd):

2. Q = {S0e, S0o, S1e, S1o, S2e, S2o, S3+e, S3+o}

3. Start: S0e
4. Accepting: S3+o
5. Transitions on 'a': advance a-count (but stay in odd/even parity)

6. Transitions on 'b': toggle parity (but stay in same a-count)

7. **Convert to right-linear grammar**:

```
S0e → a S1e | b S0o
S0o → a S1o | b S0e
S1e → a S2e | b S1o
S1o → a S2o | b S1e
S2e → a S3+e | b S2o
S2o → a S3+o | b S2e
S3+e → a S3+e | b S3+o
S3+o → a S3+o | b S3+e | ε      // ε since S3+o is accepting
```

1. **Test**: w = "aababb": na=3, nb=3 ⇒ should reach S3+o and accept.

13.3 Example 2: CFG for L = { a^n b^m : n = m - 2 }

Goal: m = n + 2. Every `a` pairs with one `b`, plus two extra `b`s at the end.

Grammar G = ({S, B}, {a,b}, P, S) with productions:

```
S → a S b      // match one 'a' with one 'b'
S → B          // when no more 'a's, go to base case
B → b b        // supply the extra two 'b's
```

Step-by-step derivation for n=2 (so m=4), producing the string `a a b b b b`:

1. Start: S
2. Apply `S → a S b`:

   ```
   S ⇒ a S b
   ```

3. Again apply `S → a S b` on the middle S:

```
a S b ⇒ a (a S b) b = a a S b b
```

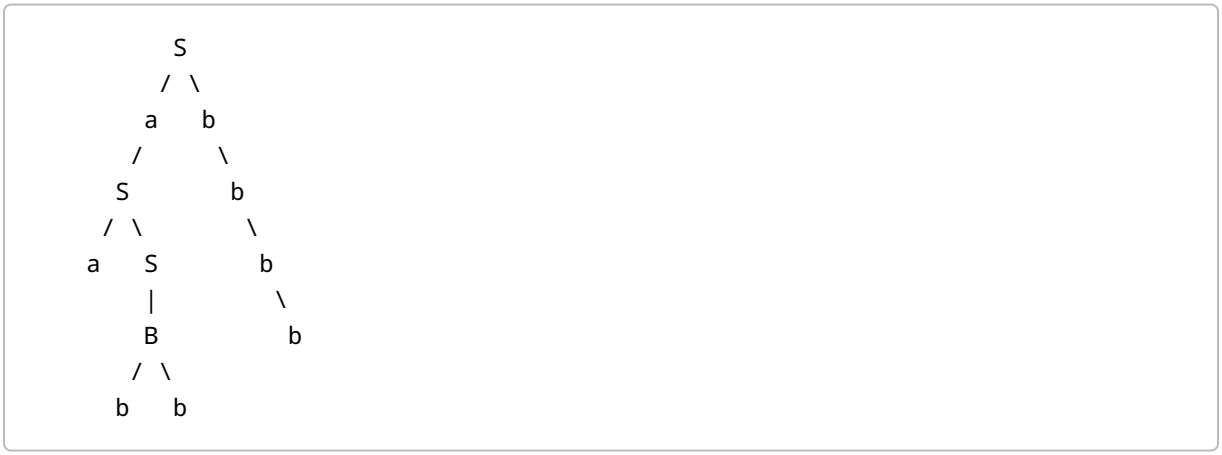
4. Now replace the remaining S with B via `S → B`:

```
a a S b b ⇒ a a B b b
```

5. Finally apply `B → b b`:

```
a a B b b ⇒ a a b b b b
```

Parse tree sketch:



This confirms that the grammar generates exactly `a^n b^(n+2)` strings.

Why it works:

- Each application of `S → a S b` consumes one `a` and one `b`, keeping track of paired symbols.
- Once all `a`'s are used, `S → B` adds the two extra `b`s.

Additional regex syntax reminder::**

- **Union**: `r₁ | r₂` means either r_1 or r_2
- **Concatenation**: `r₁r₂` means r_1 followed by r_2
- **Kleene star**: `r*` zero or more r's
- **Plus**: `r+` one or more r's
- **Optional**: `r?` zero or one r
- **Character classes**: `[abc]`, `[^0-9]`
- **Grouping**: `( … )` to form subexpressions

(Extracted and extended)")]}